



PYTHON CODING LOGIC

150 Practice Questions

Beginner → Advanced | Input/Output · Loops · Strings · DSA · OOP · Interviews

By Dr. Amar Panchal | Career Credentials





SECTION 1 — Input & Output (Q1–Q10)

◆ Beginner

1. Take your name as input and print: Hello, [name]! Welcome to coding.
2. Take two numbers as input and print their sum, difference, product, and quotient.
3. Input your birth year and print your current age.
4. Take radius as input and print area and circumference of a circle (use `math.pi`).
5. Input three subject marks and print the total and average.

◆ Intermediate

6. Take temperature in Celsius and convert it to both Fahrenheit and Kelvin.
7. Input a number and print whether it is positive, negative, or zero — without using if-else.
8. Take two numbers and swap them without using a third variable, then print.
9. Input your full name (first + last) in one line, split it, print first and last separately.
10. Input a number and print its square, cube, and square root formatted to 2 decimal places.

 **Mentor Tip:** Always validate user input. What happens if the user enters letters when you expect numbers? Try wrapping input in try-except as you advance.





SECTION 2 — Conditionals (Q11–Q20)

◆ Beginner

11. Check if a number is even or odd.
12. Input three numbers and print the largest without using max().
13. Check if a year is a leap year.
14. Input marks and print grade: A (90+), B (75+), C (60+), D (45+), F (below 45).
15. Check if a number is divisible by both 3 and 5.

◆ Intermediate

16. Input three sides of a triangle; check if it's valid, then classify it (equilateral / isosceles / scalene).
17. Build a simple calculator: input two numbers and operator (+, -, *, /), print result.
18. Input a character; check if it's a vowel, consonant, digit, or special character.
19. Movie tickets cost Rs.200 for adults and Rs.120 for children (below 12). Input age and quantity, print total.
20. Input a number and check if it's a perfect square without using sqrt.

 **Mentor Tip:** Nested if-else is fine for now, but as you progress, look for ways to flatten your conditions — it makes code easier to read and debug.



 SECTION 3 — Loops (Q21–Q40)**◆ Beginner**

21. Print numbers 1 to 100 using a loop.
22. Print all even numbers between 1 and 50.
23. Print the multiplication table of any user-inputted number.
24. Find the sum of all digits in a number (e.g., 1234 → 10).
25. Count how many times digit 7 appears in a given number.

◆ Intermediate

26. Print all prime numbers between 1 and 100.
27. Reverse a number without converting it to a string (e.g., 1234 → 4321).
28. Check if a number is a palindrome using a loop.
29. Print the Fibonacci sequence up to N terms.
30. Find the factorial of a number using a loop.
31. Print a right-angled star pattern of N rows.
32. Print a number pyramid (1 / 1 2 / 1 2 3 ...).
33. Find the GCD of two numbers using a loop.
34. Find all Armstrong numbers up to 1000 (sum of cubes of digits equals the number).
35. FizzBuzz: print 1–100, replace multiples of 3 with Fizz, 5 with Buzz, both with FizzBuzz.

◆ Advanced

36. Print a diamond star pattern of N rows.
37. Find all perfect numbers between 1 and 10000.
38. Check if a number is a Happy Number using a loop.
39. Print Pascal's Triangle up to N rows.
40. Generate all prime factors of a given number.

 **Mentor Tip:** Loops are the heart of programming logic. Master the while loop fully — not just for loops. Every for loop can be written as a while loop.





SECTION 4 — Strings (Q41–Q60)

◆ Beginner

41. Reverse a string.
42. Check if a string is a palindrome.
43. Count the number of vowels in a string.
44. Convert a sentence to uppercase, lowercase, and title case.
45. Count the total number of words in a sentence.

◆ Intermediate

46. Count the frequency of each character in a string.
47. Remove all spaces from a string.
48. Replace all vowels in a string with *.
49. Check if two strings are anagrams of each other.
50. Find the longest word in a sentence.
51. Reverse each word in a sentence (e.g., 'hello world' → 'olleh dlrow').
52. Check if a string contains only digits, only alphabets, or is mixed.
53. Remove duplicate characters from a string while preserving order.
54. Find the most frequently occurring character in a string.

◆ Advanced

55. Input a paragraph and print word frequency sorted from most to least common.
56. Compress a string using run-length encoding (e.g., 'aaabbc' → 'a3b2c1').
57. Check if one string is a rotation of another.
58. Find all permutations of a given string (without libraries).
59. Extract all email addresses from a long text string.
60. Given a sentence, capitalize every alternate word.

 **Mentor Tip:** Never use loops to build strings character-by-character in Python — use `.join()`. This habit saves enormous time on large data.





SECTION 5 — Lists (Q61–Q75)

◆ Beginner

61. Create a list of 10 numbers and print max, min, and sum without built-in functions.
62. Remove duplicates from a list while preserving order.
63. Sort a list in ascending and descending order without using `.sort()`.
64. Merge two lists and remove duplicates.
65. Find the second largest element in a list.

◆ Intermediate

66. Rotate a list by K positions to the right.
67. Flatten a nested list: `[[1,2],[3,4],[5]] → [1,2,3,4,5]`.
68. Partition a list into even numbers and odd numbers.
69. Find all pairs in a list whose sum equals a given target.
70. Remove all negative numbers from a list using list comprehension.
71. Count how many elements in a list are greater than the average.
72. Move all zeros in a list to the end while maintaining order of non-zero elements.

◆ Advanced

73. Sort student names by their corresponding marks (two separate lists) — highest to lowest.
74. Find the most frequently occurring element in a list without using Counter.
75. Given a list of integers, find the length of the longest increasing subsequence.

 **Mentor Tip:** List comprehension is powerful but don't overuse it. If a comprehension is hard to read, a regular loop is better — readable code > clever code.



 SECTION 6 — Tuples (Q76–Q82)**◆ Beginner**

- 76. Create a tuple of 5 cities and print each using a loop.
- 77. Find the maximum and minimum element in a tuple without max()/min().
- 78. Convert a tuple to a list, add an element, and convert back to a tuple.

◆ Intermediate

- 79. Unpack a tuple of (name, age, city) and print a formatted sentence.
- 80. Given a list of tuples [(name, marks)], find the student with highest marks.
- 81. Swap two variables using tuple packing/unpacking in a single line.

◆ Advanced

- 82. Sort a list of tuples by the second element (marks) in descending order without sorted() key.

 **Mentor Tip:** Tuples are immutable — use them for data that should NOT change (coordinates, RGB values, DB records). This prevents accidental modification bugs.



 SECTION 7 — Sets (Q83–Q90)**◆ Beginner**

- 83. Create two sets and find their union, intersection, and difference.
- 84. Check if one set is a subset of another.
- 85. Remove duplicates from a list using a set, then convert back to a list.

◆ Intermediate

- 86. Find common elements among three lists using sets.
- 87. Given a sentence, find all unique words using a set.
- 88. Find the symmetric difference between two sets.

◆ Advanced

- 89. 30 students took exams. Find who passed both Maths and Science, only Maths, only Science, and failed both.
- 90. Check if two strings are anagrams using sets and character frequency dictionaries combined.

 **Mentor Tip:** Sets give $O(1)$ lookup — always use a set instead of a list when you're only checking membership (e.g., 'is x in visited?'). This is a key interview technique.



 SECTION 8 — Dictionaries (Q91–Q100)◆ **Beginner**

91. Create a dictionary of 5 students with their marks. Print names of students who scored above 75.
92. Count the frequency of each word in a sentence using a dictionary.
93. Merge two dictionaries. If a key exists in both, add the values.

◆ **Intermediate**

94. Invert a dictionary (swap keys and values).
95. Find the key with the maximum value in a dictionary.
96. Group a list of words by their first letter using a dictionary.
97. Build a phone book: support add, search, update, and delete operations via a menu loop.

◆ **Advanced**

98. Given student names and marks as a dictionary, print a ranked leaderboard.
99. Count character frequency in a string; display only characters appearing more than twice, sorted alphabetically.
100. Given a list of transactions with category and amount, compute total spent per category using a dictionary.

 **Mentor Tip:** `dict.get(key, default)` is safer than `dict[key]` — it never raises a `KeyError`. Use it whenever a key might not exist.



SECTION 9 — Functions (Q101–Q115)

◆ Beginner

- 101. Write a function `is_prime(n)` that returns True if `n` is prime, else False.
- 102. Write a function `factorial(n)` using recursion.
- 103. Write a function that takes a list and returns it sorted (ascending) without built-in sort.
- 104. Write a function that returns the reverse of a string.
- 105. Write a function that takes a name and age and prints a greeting sentence.

◆ Intermediate

- 106. Write a function `is_palindrome(s)` that works for both strings and numbers.
- 107. Write a function using `*args` that takes any number of numbers and returns their average.
- 108. Write a function `fibonacci(n)` that returns the Nth Fibonacci number using recursion.
- 109. Write a function `word_frequency(sentence)` that returns a dictionary of word counts.
- 110. Write a higher-order function `apply_twice(f, x)` that applies function `f` to `x` twice.

◆ Advanced

- 111. Write a decorator `@timer` that prints how long a function takes to run.
- 112. Write a memoized version of `fibonacci(n)` using a dictionary cache.
- 113. Write a generator function that yields prime numbers up to `N`.
- 114. Write a function using `**kwargs` that builds and returns a student profile dictionary.
- 115. Build a Student Report System: `add_student()`, `get_average()`, `get_topper()`, `print_report()`.

 **Mentor Tip:** Functions should do ONE thing. If you find yourself writing a function that does three things, split it into three functions. This is the Single Responsibility Principle.



SECTION 10 — Object-Oriented Programming (Q116–Q135)

◆ Beginner

- 116. Create a class Car with attributes brand, model, and year. Add a method display_info().
- 117. Create a class Rectangle with length and width. Add methods area() and perimeter().
- 118. Create a class Student with name and marks. Add a method is_pass() that returns True if marks ≥ 35 .
- 119. Create a class BankAccount with balance. Add methods deposit(amount) and withdraw(amount).
- 120. Create a class Circle. Use `__init__` to set radius, and add methods area() and circumference().

◆ Intermediate — Inheritance

- 121. Create a base class Animal with a method speak(). Create Dog and Cat subclasses that override speak().
- 122. Create a class Shape with method area(). Inherit it in Triangle, Rectangle, and Circle. Demonstrate polymorphism.
- 123. Create a class Employee and a subclass Manager that adds a bonus attribute and overrides a salary() method.
- 124. Use `super()` to call the parent class `__init__` in a child class. Demonstrate with a Vehicle → ElectricCar hierarchy.
- 125. Create a class Person. Inherit it in Student (adds GPA) and Teacher (adds subject). Print combined info.

◆ Intermediate — Encapsulation & Properties

- 126. Create a class with a private attribute `__balance`. Use getter and setter methods to access it safely.
- 127. Use `@property` and `@setter` decorators to manage a temperature attribute that auto-converts to Fahrenheit.
- 128. Build a class ATM that enforces minimum withdrawal of Rs.100 and maximum of Rs.10000 using encapsulation.

◆ Advanced

- 129. Implement a Stack class using a list with push(), pop(), peek(), is_empty(), and size() methods.
- 130. Implement a Queue class using a list with enqueue(), dequeue(), front(), and is_empty() methods.
- 131. Create a class Library with a list of books. Add methods add_book(), remove_book(), search_book(), and list_books().



132. Use dunder methods (`__str__`, `__repr__`, `__add__`, `__len__`) in a Vector class to support vector addition and printing.

133. Create an abstract base class Payment with an abstract method `process()`. Implement CreditCard, UPI, and Cash subclasses.

134. Build a mini Inventory system using OOP: add product, remove product, update stock, display all products.

135. Implement multiple inheritance: class FlyingFish inherits from both Bird and Fish. Resolve MRO conflicts.

 **Mentor Tip:** OOP is not just about classes — it's about modeling the real world. Before writing code, always sketch: What are the objects? What do they know? What can they do?



 SECTION 11 — Interview Questions (Q136–Q150)**Coding Logic Questions (commonly asked in tech interviews)****◆ Beginner Level**

- 136.** Two Sum Problem: Given a list of numbers and a target, find two numbers that add up to the target.
- 137.** Find the missing number in a list of 1 to N with one number missing.
- 138.** Count the number of occurrences of each element in a list without using Counter.
- 139.** Check if a given string has balanced parentheses using a stack (list).
- 140.** Reverse a list in-place without using reverse() or slicing.

◆ Intermediate Level

- 141.** Remove duplicates from a sorted list in $O(n)$ without extra space.
- 142.** Given a list, find the subarray (contiguous) with the maximum sum (Kadane's Algorithm).
- 143.** Given a list of 0s, 1s, and 2s, sort it in one pass without using sort() (Dutch National Flag problem).
- 144.** Find the longest substring without repeating characters in a given string.
- 145.** Given two sorted lists, merge them into one sorted list without using sort().

◆ Advanced Level

- 146.** Implement binary search on a sorted list. Return the index or -1 if not found.
- 147.** Given a matrix (list of lists), rotate it 90 degrees clockwise in-place.
- 148.** Find all unique combinations of numbers from a list that sum to a target (backtracking).
- 149.** Implement a simple LRU Cache using a dictionary and a doubly linked list (or use OrderedDict).
- 150.**  **BOSS CHALLENGE:** Build a complete Student Management System using OOP with file-based persistence: add/update/delete/search students, compute rankings, and export a report.

**Your Learning Roadmap**

Input/Output → Conditionals → Loops → Strings
→ Lists / Tuples / Sets / Dicts → Functions → OOP → Interviews



Don't skip questions. Solve each one yourself first, then look for better solutions.

That's how real programmers grow. 

